

Dados relacionais e operações do tipo join

Isabela Diniz Cia

Introdução

Conjuntos de dados estruturados costumam ser divididos em múltiplas tabelas. A razão para isso é a minimização de duplicação de informação. Por este motivo, uma tarefa habitualmente realizada em manipulação de bases de dados é a combinação de dados de diferentes origens.

Objetivos

Ao fim deste laboratório, você deve ser capaz de combinar duas tabelas de dados, de forma que, na tabela resultante, sejam mantidos registros que existam:

- em ambas as tabelas;
- apenas na primeira tabela;
- apenas na segunda tabela;
- em alguma das duas tabelas.

Adicionalmente, você deve ser capaz de identificar registros que existam:

- apenas na primeira tabela;
- apenas na segunda tabela;

Atrasos de vôos

Considere novamente o problema de atrasos de vôos, disponível em <https://www.kaggle.com/us-dot/flight-delays>. Nesta atividade, além dos dados de `flights.csv`, nós iremos utilizar informações disponíveis nos arquivos `airlines.csv` e `airports.csv`.

1. Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.

- Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE;
- Para o arquivo de vôos
 - a. Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY;
 - b. Leia apenas 1 milhão de linhas por vez;
 - c. Remova vôos em que o aeroporto de destino comece com a letra 1;
 - d. Remova registros em que existam pelo menos uma coluna faltante;
 - e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
 - f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
library(readr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Importação dos aeroportos

Inicialmente, foi realizada a importação do arquivo `airports.csv`. Conforme solicitado no enunciado, foram selecionadas apenas as colunas IATA_CODE, CITY, STATE, LATITUDE e LONGITUDE.

```
airports <- read_csv("airports.csv",
                     col_select = c(
                       "IATA_CODE",
                       "CITY",
                       "STATE",
                       "LATITUDE",
                       "LONGITUDE"))
```

```

Rows: 322 Columns: 5
-- Column specification -----
Delimiter: ","
chr (3): IATA_CODE, CITY, STATE
dbl (2): LATITUDE, LONGITUDE

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

```

```
head(airports)
```

```

# A tibble: 6 x 5
  IATA_CODE CITY      STATE LATITUDE LONGITUDE
  <chr>      <chr>    <chr>   <dbl>   <dbl>
1 ABE      Allentown PA       40.7    -75.4
2 ABI      Abilene    TX       32.4    -99.7
3 ABQ      Albuquerque NM       35.0   -107.
4 ABR      Aberdeen  SD       45.4    -98.4
5 ABY      Albany    GA       31.5    -84.2
6 ACK      Nantucket  MA       41.3    -70.1

```

Processamento dos voos

A função `getStats_1`, realiza as seguintes etapas: 1. Remoção dos voos cujo aeroporto de destino começa com o número 1; 2. Remoção dos registros que possuem valores ausentes; 3. Agrupamento dos voos por aeroporto de destino; 4. Cálculo da soma dos atrasos e do número total de voos.

- Observação: A soma dos atrasos e o número de voos são estatísticas suficientes para que, após a leitura de todas as partes do arquivo, seja possível calcular o atraso médio de cada aeroporto.

```

getStats_1 <- function(input, pos) {
  input %>%
    filter(substr(DESTINATION_AIRPORT, 1, 1) != "1") %>%
    filter(!is.na(DESTINATION_AIRPORT), !is.na(ARRIVAL_DELAY)) %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      soma_atrasos = sum(ARRIVAL_DELAY),
      total = n(),
      .groups = 'drop'
    )
}

```

```
)
}
```

Como o arquivo `flights.csv` possui um grande volume de dados, sua leitura foi realizada em partes de até 1 milhão de linhas e a cada bloco foi aplicada cada etapa da função `getStats_1`.

```
flights <- read_csv_chunked(
  'flights.csv',
  callback = DataFrameCallback$new(getStats_1),
  chunk_size = 1000000,
  col_types = cols_only(
    DESTINATION_AIRPORT = 'c',
    ARRIVAL_DELAY = 'd'
  )
)
```

Após o processamento de todas as partes do arquivo, cada aeroporto pode aparecer em mais de um bloco de dados.

Portanto, é necessário combinar as estatísticas obtidas em cada parte. Para isso, somamos:

- a soma dos atrasos;
- o número total de voos.

Finalmente, o atraso médio é obtido por:

```
flights <- flights %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(
    soma_atrasos = sum(soma_atrasos),
    total = sum(total),
    .groups = 'drop'
  ) %>%
  mutate(
    media_atraso = soma_atrasos / total
  )

head(flights)
```

```
# A tibble: 6 x 4
  DESTINATION_AIRPORT soma_atrasos total media_atraso
  <chr>                <dbl> <int>      <dbl>
```

1	ABE	12874	2220	5.80
2	ABI	9077	2224	4.08
3	ABQ	106415	18953	5.61
4	ABR	-2227	657	-3.39
5	ABY	7501	864	8.68
6	ACK	2393	487	4.91

2. Selecione a operação apropriada join para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

a. Identifique a coluna que é a chave na tabela `flights`;

- **Resposta:** A chave utilizada na tabela `flights` é a coluna `DESTINATION_AIRPORT`, pois ela identifica o código do aeroporto de destino de cada registro agregado.

b. Identifique a coluna que é a chave na tabela `airports`;

- **Resposta:** A chave utilizada na tabela `airports` é a coluna `IATA_CODE`, pois ela contém o código identificador de cada aeroporto.

c. Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`?

`anti_join`

```
flights %>%
  anti_join(
    airports,
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  )
```

```
# A tibble: 0 x 4
#   i 4 variables: DESTINATION_AIRPORT <chr>, soma_atrasos <dbl>, total <int>,
#   media_atraso <dbl>
```

`setdiff(x, y)`

```
setdiff(
  flights$DESTINATION_AIRPORT,
  airports$IATA_CODE)
```

`character(0)`

Os resultados obtidos por meio de `anti_join()` e `setdiff()` confirmam que não existem aeroportos presentes na tabela `flights` que estejam ausentes na tabela `airports`.

- d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
- e. Armazene a tabela resultante no objeto `flights`.

Para combinar as informações das tabelas, foi utilizado `left_join()`, mantendo todos os registros existentes em `flights`.

Embora inicialmente o exercício solicite a inclusão das colunas `CITY` e `STATE`, as colunas `LATITUDE` e `LONGITUDE` também são mantidas, pois serão necessárias posteriormente para a construção do mapa.

```
flights <- flights %>%
  left_join(
    airports,
    by = c(
      "DESTINATION_AIRPORT" = "IATA_CODE"
    )
  )

head(flights)
```

```
# A tibble: 6 x 8
  DESTINATION_AIRPORT soma_atrasos total media_atraso CITY        STATE LATITUDE
  <chr>                <dbl> <int>         <dbl> <chr>      <chr>    <dbl>
1 ABE                  12874  2220          5.80 Allentown  PA        40.7
2 ABI                   9077  2224          4.08 Abilene    TX        32.4
3 ABQ                106415 18953          5.61 Albuquerque NM        35.0
4 ABR                  -2227   657         -3.39 Aberdeen  SD        45.4
5 ABY                   7501   864          8.68 Albany    GA        31.5
6 ACK                   2393   487          4.91 Nantucket  MA        41.3
# i 1 more variable: LONGITUDE <dbl>
```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
aeroporto_estado <- flights %>%
  distinct(DESTINATION_AIRPORT, STATE) %>%
  group_by(STATE) %>%
  summarise(
    n_aeroportos = n(),
    .groups = 'drop'
```

```
) %>%
  arrange(desc(n_aeroportos))

head(aeroporto_estado)
```

```
# A tibble: 6 x 2
  STATE n_aeroportos
  <chr>     <int>
1 TX             24
2 CA             22
3 AK             19
4 FL             17
5 MI             15
6 NY             14
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

a. Carregue o pacote `leaflet`;

```
library(leaflet)
```

b. Combine os comandos `leaflet`, `addTiles` e `addMarkers` para a criação de um mapa básico;

c. Armazene o grafico em b) numa variável chamada `this`;

```
#flights <- flights %>%
#   filter(!is.na(LATITUDE), !is.na(LONGITUDE))

#this <- leaflet(flights) %>%
#   addTiles() %>%
#   addMarkers(
#     lng = ~LONGITUDE,
#     lat = ~LATITUDE,
#     popup = ~paste(DESTINATION_AIRPORT,
#                     "| Estado:",
#                     STATE,
#                     "| Atraso médio:",
#                     round(media_atraso, 2),
#                     "minutos")
#   )

# this
```

Horário de conclusão

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 20:05:04 -03"
```